

Phase IV: Software Testing

TESS – Tirana Employment Service for Students

Team SmartShift – Group 5

Task Overview: Choose 2-3 critical classes, or functions from your software, prepare comprehensive test cases, write the related test code, execute the tests, and document the results clearly in your report.

1. Introduction to Testing

Testing is known as the process of exercising a program with the specific purpose of finding errors, so that a decent version of the system can be delivered to the end user. Software testing is also defined as the whole process of evaluating a system with the aim of identifying defects, bugs or unexpected behavior before the system is deployed. Software testing may include the execution of a program or other component, verifying that it all behaves as expected and meets the defined requirements.

Testing is a critical activity in software development for multiple reasons. Testing helps with bug detection, identifying defects early in the development process since the costs of fixing are less during that phase. Second, it ensures reliability and correctness by verifying that software components behave as intended under both expected and unexpected conditions. Third, it confirms compliance of the system by systematically checking all important paths through the code and ensuring that software adheres to the specified requirements. Finally, well-tested code is easier to maintain because developers can confidently make changes knowing that tests will catch regressions.

2. Purpose of Testing

Software testing serves several fundamental purposes in the software development process.

Defect Detection: Testing helps identify defects and bugs early in the development cycle, before the software reaches end users. Detecting issues early is significantly less costly than finding them after deployment, as fixes require less effort and cause less disruption.

Behavioral Verification: Testing verifies that software components behave as intended under both expected and unexpected conditions. This includes checking normal inputs, boundary values and invalid inputs to ensure the system responds correctly in every scenario.

Reliability and Correctness: Testing improves the overall reliability of the software by systematically checking all important execution paths. A well-tested system gives developers and users confidence that the software will perform consistently under real-world conditions.

Maintainability: Automated tests act as a safety net when making changes to the codebase. When a developer modifies existing code or adds new features, the tests can be re-run to immediately detect if anything has broken. This is known as regression testing.

Security Validation: Testing verifies that security mechanisms such as authentication, authorization, and input validation work correctly and cannot be bypassed by invalid or malicious inputs.

Documentation: Well-written test cases also serve as living documentation of how the system is expected to behave, making it easier for new developers to understand the codebase.

3. Focus on Testing a Single Component

We selected three critical components from the TESS backend for testing:

*Component 1: **JobMatchingService***

This is the most important and complex component in the entire system, serving as the core innovation of TESS the algorithm that automatically filters and ranks job opportunities based on a student's academic schedule and skills. If this component contains a bug, students may see jobs that conflict with their classes or miss jobs they are compatible with. It contains multiple methods, conditional logic and edge cases.

*Component 2: **AuthService (Registration)***

The registration component controls who enters the system and validates critical user data such as age constraints, required fields and email uniqueness. Security and data integrity depend on this component working correctly. Errors here could allow underage users, fake employers or duplicate accounts to enter the system.

Component 3: StudentService (Availability Schedule Management)

The availability schedule is the foundation of the matching algorithm. If schedules are saved incorrectly, for example with invalid time ranges or belonging to the wrong student, then the entire matching system would break down. This component also implements an ownership check that is critical for data security.

4. Preparing Test Cases

The test cases were designed to cover different possible Job matching service; StudentService Availability cases; as well as possible AuthService Registration scenarios.

Framework: JUnit 5 (Jupiter): the standard Java testing framework, included in Spring Boot Test

Mocking: Mockito: used to mock database repositories and security context so tests run without a real database

Assertions: AssertJ: assertion library that provides readable and expressive test assertions

Test Runner: Maven Surefire Plugin: executes tests automatically with `mvn test` command

Regarding the setup all test files are located in `src/test/java/com/tess/`. The tests use the `@ExtendWith(MockitoExtension.class)` annotation which initializes mocks automatically. No external database or server is required, since all dependencies are mocked using `@Mock` and `@InjectMocks`.

To run the tests, we execute in the project terminal: mvn test command.

Component 1: JobMatchingService — 15 Test Cases

Test ID	Scenario	Input	Expected Result
TC01	No conflict: job shift after class	Student busy MON 09-12, Job MON 14-18	No conflict detected
TC02	Direct overlap conflict	Student busy MON 09-12, Job MON 09-13	Conflict detected
TC03	Partial overlap	Student busy MON 10-14, Job MON 13-17	Conflict detected
TC04	Back-to-back times	Student busy MON 10-14, Job MON 13-17	No conflict (exact boundary)
TC05	Different day	Student busy MON 09-12, Job TUE 09-13	No conflict
TC06	Job has no schedule	Job with empty workDays, any busy slot	No conflict
TC07	Skill match exists	Job needs Java, Student has Java	Skill match true
TC08	No skill match	Job needs Excel, Student has Java	Skill match false
TC09	Case insensitive skill	Job needs "Java", Student has "java"	Skill match true
TC10	Empty skills list	Job needs Java, Student has no skills	Skill match false
TC11	No schedule set	Students have no busy slots	All active jobs returned
TC12	Conflicting job filtered	One job conflicts, one does not	Only non-conflicting job returned
TC13	Skill-matched ranked first	Two compatible jobs, one skill match	Skill-matched job first in results
TC14	Perfect compatibility score	No conflict + all skills match + active	Score = 100

Test ID	Scenario	Input	Expected Result
TC15	Low score with conflict	Schedule conflict exists	Score $\leq$ 50

Component 2: AuthService Registration — 11 Test Cases

Test ID	Scenario	Input	Expected Result
TC01	Valid student registration	All valid student fields	Registration succeeds
TC02	Duplicate email	Email already in database	BadRequestException: "Email is already in use"
TC03	Age too young	Student age = 14	BadRequestException: "age must be between 16 and 35"
TC04	Age too old	Student age = 40	BadRequestException: "age must be between 16 and 35"
TC05	Minimum age boundary	Student age = 16	Registration succeeds
TC06	Maximum age boundary	Student age = 35	Registration succeeds
TC07	Missing university	University = null	BadRequestException: "University is required"
TC08	Not an active student	activeStudent = false	BadRequestException: "must be an active student"
TC09	Valid employer registration	All valid employer fields	Registration succeeds
TC10	Missing company name (null)	companyName = null	BadRequestException: "Company name is required"
TC11	Blank company name	companyName = " "	BadRequestException: "Company name is required"

Component 3: StudentService Availability — 9 Test Cases

Test ID	Scenario	Input	Expected Result
TC01	Valid slot saved	MON 09:00-12:00, busy=true	Slot saved with correct fields
TC02	End before start	MON 14:00-10:00	BadRequestException: "End time must be after start time"
TC03	Multiple slots	3 slots on different days	All 3 slots saved
TC04	Age too old	Student age = 40	BadRequestException: "age must be between 16 and 35"
TC05	Old slots deleted	Save new schedule	deleteByStudentId() called before saving
TC06	Empty schedule	Save empty list	All slots cleared, result is empty
TC07	Ownership violation	Different user tries to edit	AccessDeniedException: "only modify your own profile"
TC08	isBusy=true saved	Slot with busy=true	isBusy field is true in saved entity
TC09	isBusy=false saved	Slot with busy=false	isBusy field is false in saved entity

5. Writing Test Code

Component 1: *JobMatchingService* — Key Test Methods

```
// TC01 - No schedule conflict: job afternoon, student busy morning
@Test
@DisplayName("TC01 - No conflict when job shift is after student class")
void tc01_noConflict_whenJobShiftAfterClass() {
    // Student busy MONDAY 09:00-12:00
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(12, 0))
        .isBusy(true)
        .build();

    // Job MONDAY 14:00-18:00
    boolean conflict = jobMatchingService.hasScheduleConflict(
        jobMondayAfternoon, List.of(busySlot));
    assertThat(conflict).isFalse();
}

// TC02 - Conflict: job overlaps with student class
@Test
@DisplayName("TC02 - Conflict detected when job overlaps with class")
void tc02_conflict_whenJobOverlapsWithClass() {
    // Student busy MONDAY 09:00-12:00
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(12, 0))
        .isBusy(true)
        .build();

    // Job MONDAY 09:00-13:00 - overlaps
    boolean conflict = jobMatchingService.hasScheduleConflict(
        jobMondayMorning, List.of(busySlot));
    assertThat(conflict).isTrue();
}
```

```
// TC08 - No skill match: student has none of the required skills
@Test
@DisplayName("TC08 - Skill match returns false when student has no required skill")
void tc08_skillMatchFalse() {
    // Job requires Excel, Typing
    // Student has Java, React, SQL
    boolean match = jobMatchingService.hasSkillMatch(
        jobMondayMorning, List.of("Java", "React", "SQL"));
    assertThat(match).isFalse();
}

// TC09 - Case insensitive skill matching
@Test
@DisplayName("TC09 - Skill match is case insensitive")
void tc09_skillMatchCaseInsensitive() {
    // Job requires "Java" (capitalized)
    // Student has "java" (lowercase)
    boolean match = jobMatchingService.hasSkillMatch(
        jobMondayAfternoon, List.of("java", "react"));
    assertThat(match).isTrue();
}

// TC10 - Empty student skills: no skill match
@Test
@DisplayName("TC10 - No skill match when student has no skills")
void tc10_noSkillMatchWhenStudentHasNoSkills() {
    boolean match = jobMatchingService.hasSkillMatch(
        jobMondayAfternoon, List.of());
    assertThat(match).isFalse();
}
```

```
// TC04 - Back-to-back times: no overlap (class ends at 12, job starts at 12)
@Test
@DisplayName("TC04 - Back-to-back times do not conflict")
void tc04_backToBackTimesNoConflict() {
    // Student busy MONDAY 09:00-12:00
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(12, 0))
        .isBusy(true)
        .build();

    // Job MONDAY 12:00-16:00 - starts exactly when class ends
    Job job = new Job();
    job.setWorkDays(List.of(AvailabilityDay.MONDAY));
    job.setShiftStartTime(LocalTime.of(12, 0));
    job.setShiftEndTime(LocalTime.of(16, 0));

    boolean conflict = jobMatchingService.hasScheduleConflict(job, List.of(busySlot));

    assertThat(conflict).isFalse();
}
```

```
// TC03 - Partial overlap still causes conflict
@Test
@DisplayName("TC03 - Partial time overlap is still a conflict")
void tc03_partialOverlapIsConflict() {
    // Student busy MONDAY 10:00-14:00
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(10, 0))
        .endTime(LocalTime.of(14, 0))
        .isBusy(true)
        .build();

    // Job MONDAY 13:00-17:00 - partial overlap at 13:00-14:00
    Job job = new Job();
    job.setWorkDays(List.of(AvailabilityDay.MONDAY));
    job.setShiftStartTime(LocalTime.of(13, 0));
    job.setShiftEndTime(LocalTime.of(17, 0));

    boolean conflict = jobMatchingService.hasScheduleConflict(job, List.of(busySlot));
    assertThat(conflict).isTrue();
}
```

```
// TC14 - Compatibility score: perfect match = 100
@Test
@DisplayName("TC14 - Perfect compatibility score is 100")
void tc14_perfectCompatibilityScore() {
    // No busy slots → no conflict (50 pts)
    // All skills match → 40 pts
    // Job active → 10 pts
    when(availabilityRepository.findById(1L)).thenReturn(List.of());
    // Job requires exactly Java which student has
    Job perfectJob = new Job();
    perfectJob.setStatus(JobStatus.ACTIVE);
    perfectJob.setWorkDays(List.of(AvailabilityDay.MONDAY));
    perfectJob.setShiftStartTime(LocalTime.of(14, 0));
    perfectJob.setShiftEndTime(LocalTime.of(18, 0));
    perfectJob.setRequiredSkills(List.of("Java")); // 1 skill, student has it
    int score = jobMatchingService.calculateCompatibilityScore(student, perfectJob);
    assertThat(score).isEqualTo(100);
}

// TC15 - Compatibility score: conflict = max 50 pts
@Test
@DisplayName("TC15 - Compatibility score is low when schedule conflict exists")
void tc15_lowScoreWhenConflict() {
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(12, 0))
        .isBusy(true)
        .build();

    when(availabilityRepository.findById(1L)).thenReturn(List.of(busySlot));
    // Job conflicts with class (MONDAY 09-13)
    int score = jobMatchingService.calculateCompatibilityScore(student, jobMondayMorning);
    // No conflict points (0) + no skill match + active (10) = max 10
    assertThat(score).isLessThanOrEqualTo(50);
}
```

```
// TC05 - Different day: no conflict
@Test
@DisplayName("TC05 - No conflict when job and class are on different days")
void tc05_differentDayNoConflict() {
    // Student busy MONDAY 09:00-12:00
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(12, 0))
        .isBusy(true)
        .build();

    // Job requires TUESDAY - different day
    Job job = new Job();
    job.setWorkDays(List.of(AvailabilityDay.TUESDAY));
    job.setShiftStartTime(LocalTime.of(9, 0));
    job.setShiftEndTime(LocalTime.of(13, 0));

    boolean conflict = jobMatchingService.hasScheduleConflict(job, List.of(busySlot));
    assertThat(conflict).isFalse();
}
```

```
// TC12 - Conflicting job filtered out from results
@Test
@DisplayName("TC12 - Conflicting job is filtered out of matching results")
void tc12_conflictingJobFilteredOut() {
    // Student busy MONDAY 09:00-12:00
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(12, 0))
        .isBusy(true)
        .build();

    when(availabilityRepository.findById(1L)).thenReturn(List.of(busySlot));
    when(jobRepository.findByStatus(eq(JobStatus.ACTIVE), any(Pageable.class)))
        .thenReturn(new PageImpl<>(List.of(jobMondayAfternoon, jobMondayMorning)));

    Page<Job> result = jobMatchingService.getMatchingJobsForStudent(student, pageable);

    // Only afternoon job passes - morning job conflicts
    assertThat(result.getContent()).hasSize(1);
    assertThat(result.getContent().get(0).getTitle()).isEqualTo("Cashier Assistant");
}
```

```
// TC13 - Skill matched jobs ranked first
@Test
@DisplayName("TC13 - Skill-matched jobs appear before non-skill-matched jobs")
void tc13_skillMatchedJobsRankedFirst() {
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.WEDNESDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(11, 0))
        .isBusy(true)
        .build();

    // Job with skill match (requires Java - student has Java)
    Job skillJob = new Job();
    skillJob.setId(10L);
    skillJob.setTitle("Java Developer");
    skillJob.setStatus(JobStatus.ACTIVE);
    skillJob.setWorkDays(List.of(AvailabilityDay.MONDAY));
    skillJob.setShiftStartTime(LocalTime.of(14, 0));
    skillJob.setShiftEndTime(LocalTime.of(18, 0));
    skillJob.setRequiredSkills(List.of("Java"));

    // Job without skill match (requires cooking - student doesn't have)
    Job noSkillJob = new Job();
    noSkillJob.setId(11L);
    noSkillJob.setTitle("Cook Assistant");
    noSkillJob.setStatus(JobStatus.ACTIVE);
    noSkillJob.setWorkDays(List.of(AvailabilityDay.TUESDAY));
    noSkillJob.setShiftStartTime(LocalTime.of(14, 0));
    noSkillJob.setShiftEndTime(LocalTime.of(18, 0));
    noSkillJob.setRequiredSkills(List.of("Cooking"));
    when(availabilityRepository.findById(1L)).thenReturn(List.of(busySlot));
    when(jobRepository.findByStatus(eq(JobStatus.ACTIVE), any(Pageable.class)))
        .thenReturn(new PageImpl<>(List.of(noSkillJob, skillJob)));

    Page<Job> result = jobMatchingService.getMatchingJobsForStudent(student, pageable);
    // skillJob should be first even though noSkillJob was first in input
    assertThat(result.getContent()).hasSize(2);
    assertThat(result.getContent().get(0).getTitle()).isEqualTo("Java Developer");
}
```

```
// TC06 - Job with no work days defined: no conflict possible
@Test
@DisplayName("TC06 - No conflict when job has no work days defined")
void tc06_noConflictWhenJobHasNoWorkDays() {
    Availability busySlot = Availability.builder()
        .dayOfWeek(AvailabilityDay.MONDAY)
        .startTime(LocalTime.of(9, 0))
        .endTime(LocalTime.of(12, 0))
        .isBusy(true)
        .build();

    boolean conflict = jobMatchingService.hasScheduleConflict(
        jobNoSchedule, List.of(busySlot));

    assertThat(conflict).isFalse();
}

// TC07 - Skill match: student has at least one required skill
@Test
@DisplayName("TC07 - Skill match returns true when student has a required skill")
void tc07_skillMatchTrue() {
    // Job requires Java, Communication
    // Student has Java, React, SQL
    boolean match = jobMatchingService.hasSkillMatch(
        jobMondayAfternoon, List.of("Java", "React", "SQL"));

    assertThat(match).isTrue();
}
```

Explanation: TC01 and TC02 test the core overlap formula directly. TC12 tests the full pipeline, from loading busy slots through the repository, to filtering jobs, to returning the correct subset. The `when(...).thenReturn(...)` calls are Mockito stubs that simulate database responses without a real database.

```
// TC11 - No busy slots: all active jobs returned
@Test
@DisplayName("TC11 - All active jobs returned when student has no schedule")
void tc11_allJobsReturnedWhenNoSchedule() {
    // No busy slots
    when(availabilityRepository.findById(1L)).thenReturn(List.of());
    Page<Job> allJobs = new PageImpl<>(List.of(jobMondayAfternoon, jobMondayMorning));
    when(jobRepository.findByStatus(eq(JobStatus.ACTIVE), any(Pageable.class)))
        .thenReturn(allJobs);

    Page<Job> result = jobMatchingService.getMatchingJobsForStudent(student, pageable);

    assertThat(result.getContent()).hasSize(2);
    assertThat(result.getContent()).contains(jobMondayAfternoon, jobMondayMorning);
}
```

Component 2: *AuthService* — Key Test Methods

```
// TC01 - Valid student registration succeeds
@Test
@DisplayName("TC01 - Valid student registration completes successfully")
void tc01_validStudentRegistration() {
    assertThatCode(() -> authService.register(validStudentRequest))
        .doesNotThrowAnyException();
}

// TC02 - Duplicate email rejected
@Test
@DisplayName("TC02 - Registration fails when email already exists")
void tc02_duplicateEmailRejected() {
    when(userRepository.existsByEmail("eajdinolli23@epoka.edu.al")).thenReturn(true);

    assertThatThrownBy(() -> authService.register(validStudentRequest))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("Email is already in use");
}

// TC03 - Student age too young (below 16) rejected
@Test
@DisplayName("TC03 - Registration fails when student age is below 16")
void tc03_ageTooYoungRejected() {
    validStudentRequest.setAge(14);

    assertThatThrownBy(() -> authService.register(validStudentRequest))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("age must be between 16 and 35");
}
```

```
// TC04 - Student age too old (above 35) rejected
@Test
@DisplayName("TC04 - Registration fails when student age is above 35")
void tc04_ageTooOldRejected() {
    validStudentRequest.setAge(40);

    assertThatThrownBy(() -> authService.register(validStudentRequest))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("age must be between 16 and 35");
}

// TC05 - Boundary age 16 accepted
@Test
@DisplayName("TC05 - Registration succeeds with minimum age of 16")
void tc05_minimumAgeAccepted() {
    validStudentRequest.setAge(16);

    assertThatCode(() -> authService.register(validStudentRequest))
        .doesNotThrowAnyException();
}

// TC06 - Boundary age 35 accepted
@Test
@DisplayName("TC06 - Registration succeeds with maximum age of 35")
void tc06_maximumAgeAccepted() {
    validStudentRequest.setAge(35);

    assertThatCode(() -> authService.register(validStudentRequest))
        .doesNotThrowAnyException();
}
```

```
// TC07 - Missing university rejected
@Test
@DisplayName("TC07 - Registration fails when university is missing")
void tc07_missingUniversityRejected() {
    validStudentRequest.setUniversity(null);

    assertThatThrownBy(() -> authService.register(validStudentRequest))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("University is required");
}

// TC08 - Non-active student rejected
@Test
@DisplayName("TC08 - Registration fails when student is not active")
void tc08_nonActiveStudentRejected() {
    validStudentRequest.setActiveStudent(false);

    assertThatThrownBy(() -> authService.register(validStudentRequest))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("must be an active student");
}

// TC09 - Valid employer registration succeeds
@Test
@DisplayName("TC09 - Valid employer registration completes successfully")
void tc09_validEmployerRegistration() {
    assertThatCode(() -> authService.register(validEmployerRequest))
        .doesNotThrowAnyException();
}
```

```
// TC10 - Employer missing company name rejected
@Test
@DisplayName("TC10 - Employer registration fails when company name is missing")
void tc10_missingCompanyNameRejected() {
    validEmployerRequest.setCompanyName(null);

    assertThatThrownBy(() -> authService.register(validEmployerRequest))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("Company name is required");
}

// TC11 - Blank company name rejected
@Test
@DisplayName("TC11 - Employer registration fails when company name is blank")
void tc11_blankCompanyNameRejected() {
    validEmployerRequest.setCompanyName(" ");

    assertThatThrownBy(() -> authService.register(validEmployerRequest))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("Company name is required");
}
```

Explanation: `assertThatThrownBy` verifies that the method throws the correct exception with the correct message. `assertThatCode(...). doesNotThrowAnyException()` verifies that valid input is accepted without errors. The boundary cases TC05 and TC06 are important because they verify that the age limits are inclusive (16 and 35 are both valid).

Component 3: *StudentService* — Key Test Methods

```
// TC01 - Valid availability slot saved successfully
@Test
@DisplayName("TC01 - Valid availability slot is saved successfully")
void tc01_validAvailabilitySlotsSaved() {
    AvailabilityRequest req = new AvailabilityRequest();
    req.setDayOfWeek(AvailabilityDay.MONDAY);
    req.setStartTime(LocalTime.of(9, 0));
    req.setEndTime(LocalTime.of(12, 0));
    req.setBusy(true);
    req.setDescription("Software Engineering");

    when(availabilityRepository.saveAll(anyList())).thenReturn(i -> i.getArgument(0));
    List<Availability> result = studentService.replaceAvailability(1L, List.of(req));
    assertThat(result).hasSize(1);
    assertThat(result.get(0).getDayOfWeek()).isEqualTo(AvailabilityDay.MONDAY);
    assertThat(result.get(0).getStartTime()).isEqualTo(LocalTime.of(9, 0));
    assertThat(result.get(0).getEndTime()).isEqualTo(LocalTime.of(12, 0));
    assertThat(result.get(0).isBusy()).isTrue();
}

// TC02 - End time before start time rejected
@Test
@DisplayName("TC02 - Availability rejected when end time is before start time")
void tc02_endTimeBeforeStartTimeRejected() {
    AvailabilityRequest req = new AvailabilityRequest();
    req.setDayOfWeek(AvailabilityDay.MONDAY);
    req.setStartTime(LocalTime.of(14, 0));
    req.setEndTime(LocalTime.of(10, 0)); // end before start
    req.setBusy(true);

    assertThatThrownBy(() -> studentService.replaceAvailability(1L, List.of(req)))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("End time must be after start time");
}
```

```
// TC03 - Equal start and end time rejected
@Test
@DisplayName("TC03 - Availability rejected when start and end time are equal")
void tc03_equalStartEndTimeRejected() {
    AvailabilityRequest req = new AvailabilityRequest();
    req.setDayOfWeek(AvailabilityDay.TUESDAY);
    req.setStartTime(LocalTime.of(10, 0));
    req.setEndTime(LocalTime.of(10, 0));
    req.setBusy(false);

    assertThatThrownBy(() -> studentService.replaceAvailability(1L, List.of(req)))
        .isInstanceOf(BadRequestException.class)
        .hasMessageContaining("End time must be after start time");
}
```

```
// TC05 - Old slots deleted before saving new ones
@Test
@DisplayName("TC05 - Old availability slots are deleted before saving new ones")
void tc05_oldSlotsDeletedBeforeNewOnes() {
    AvailabilityRequest req = new AvailabilityRequest();
    req.setDayOfWeek(AvailabilityDay.MONDAY);
    req.setStartTime(LocalTime.of(9, 0));
    req.setEndTime(LocalTime.of(12, 0));
    req.setBusy(true);

    when(availabilityRepository.saveAll(anyList())).thenReturn(i -> i.getArgument(0));

    studentService.replaceAvailability(1L, List.of(req));

    verify(availabilityRepository, times(1)).deleteByStudentId(1L);
}

// TC06 - Empty schedule clears all slots
@Test
@DisplayName("TC06 - Saving empty schedule clears all existing slots")
void tc06_emptyScheduleClearsAllSlots() {
    when(availabilityRepository.saveAll(anyList())).thenReturn(List.of());

    List<Availability> result = studentService.replaceAvailability(1L, List.of());

    assertThat(result).isEmpty();
    verify(availabilityRepository, times(1)).deleteByStudentId(1L);
}
```

```
// TC04 - Multiple slots saved correctly
@Test
@DisplayName("TC04 - Multiple availability slots saved correctly")
void tc04_multipleSlotsAllSaved() {
    AvailabilityRequest slot1 = new AvailabilityRequest();
    slot1.setDayOfWeek(AvailabilityDay.MONDAY);
    slot1.setStartTime(LocalTime.of(9, 0));
    slot1.setEndTime(LocalTime.of(12, 0));
    slot1.setBusy(true);

    AvailabilityRequest slot2 = new AvailabilityRequest();
    slot2.setDayOfWeek(AvailabilityDay.WEDNESDAY);
    slot2.setStartTime(LocalTime.of(14, 0));
    slot2.setEndTime(LocalTime.of(17, 0));
    slot2.setBusy(false);

    AvailabilityRequest slot3 = new AvailabilityRequest();
    slot3.setDayOfWeek(AvailabilityDay.FRIDAY);
    slot3.setStartTime(LocalTime.of(10, 0));
    slot3.setEndTime(LocalTime.of(12, 0));
    slot3.setBusy(true);

    when(availabilityRepository.saveAll(anyList())).thenReturn(i -> i.getArgument(0));

    List<Availability> result = studentService.replaceAvailability(
        1L, List.of(slot1, slot2, slot3));

    assertThat(result).hasSize(3);
}
```

```
// TC08 - isBusy flag correctly set to true
@Test
@DisplayName("TC08 - isBusy flag is correctly set to true for class slots")
void tc08_isBusyFlagSetToTrue() {
    AvailabilityRequest req = new AvailabilityRequest();
    req.setDayOfWeek(AvailabilityDay.THURSDAY);
    req.setStartTime(LocalTime.of(9, 0));
    req.setEndTime(LocalTime.of(11, 0));
    req.setBusy(true);
    req.setDescription("Database Systems");

    when(availabilityRepository.saveAll(anyList())).thenReturn(i -> i.getArgument(0));
    List<Availability> result = studentService.replaceAvailability(1L, List.of(req));

    assertThat(result.get(0).isBusy()).isTrue();
    assertThat(result.get(0).getDescription()).isEqualTo("Database Systems");
}

// TC09 - isBusy flag correctly set to false for free slots
@Test
@DisplayName("TC09 - isBusy flag is correctly set to false for free slots")
void tc09_isBusyFlagSetToFalse() {
    AvailabilityRequest req = new AvailabilityRequest();
    req.setDayOfWeek(AvailabilityDay.FRIDAY);
    req.setStartTime(LocalTime.of(14, 0));
    req.setEndTime(LocalTime.of(18, 0));
    req.setBusy(false);

    when(availabilityRepository.saveAll(anyList())).thenReturn(i -> i.getArgument(0));
    List<Availability> result = studentService.replaceAvailability(1L, List.of(req));

    assertThat(result.get(0).isBusy()).isFalse();
}
```

```
// TC07 - Ownership check: different user cannot edit
@Test
@DisplayName("TC07 - Access denied when different user tries to edit schedule")
void tc07_ownershipCheckFails() {
    // Student belongs to user ID 99, but logged-in user is ID 1
    User differentUser = new User();
    differentUser.setId(99L);
    student.setUser(differentUser);

    AvailabilityRequest req = new AvailabilityRequest();
    req.setDayOfWeek(AvailabilityDay.MONDAY);
    req.setStartTime(LocalTime.of(9, 0));
    req.setEndTime(LocalTime.of(12, 0));
    req.setBusy(true);

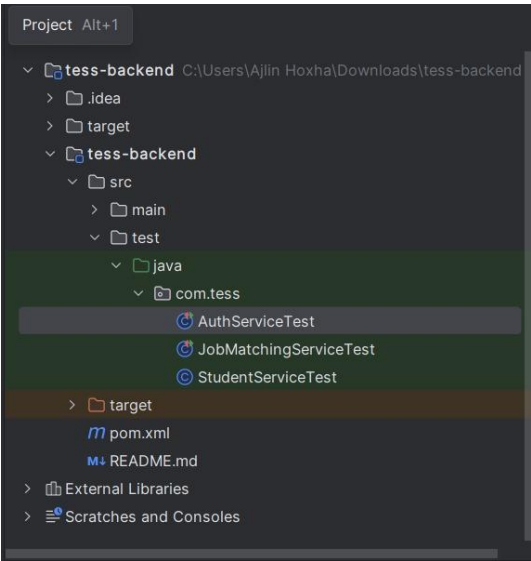
    assertThatThrownBy(() -> studentService.replaceAvailability(1L, List.of(req)))
        .isInstanceOf(AccessDeniedException.class)
        .hasMessageContaining("only modify your own profile");
}
```

Explanation: TC07 tests a security-critical feature — the ownership check. It simulates a scenario where the logged-in user (ID 1) tries to modify a student profile belonging to a different user (ID 99). The system must reject this with an `AccessDeniedException`. TC05 uses Mockito's `verify()` to confirm that the delete operation was actually called, not just that the save succeeded.

6. Running Tests

- Explain how the tests were executed.
- Describe how to interpret test results, including passing, failing, and error outcomes.
- Include screenshots, logs, or summaries if relevant.

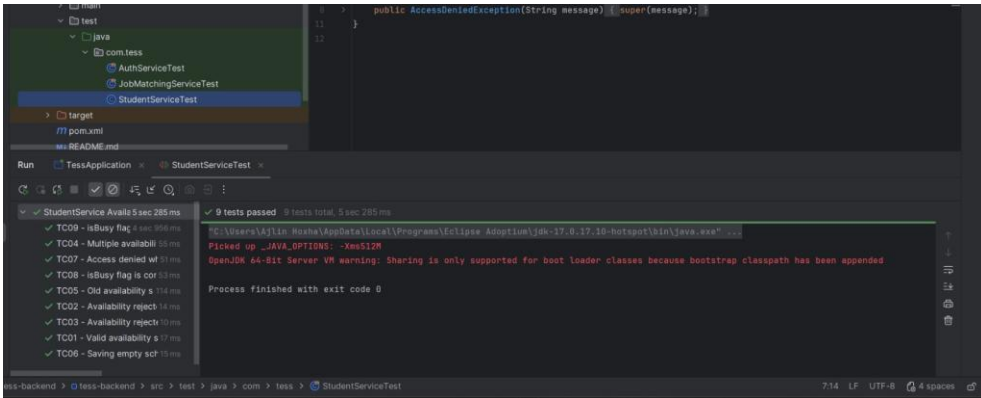
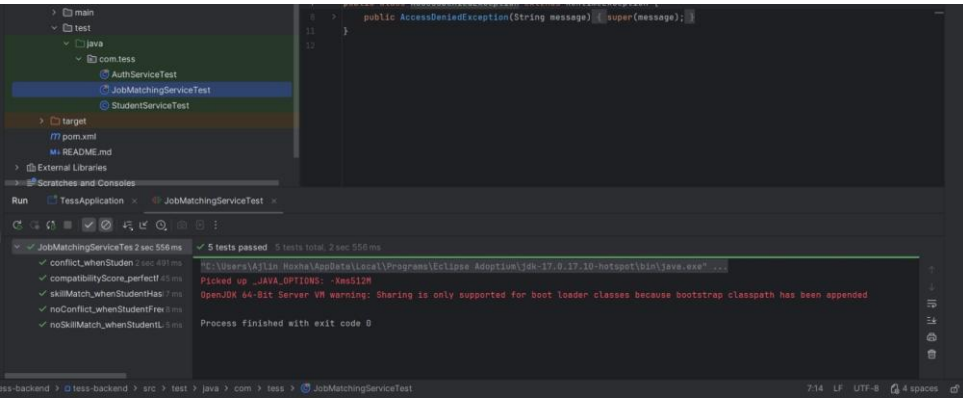
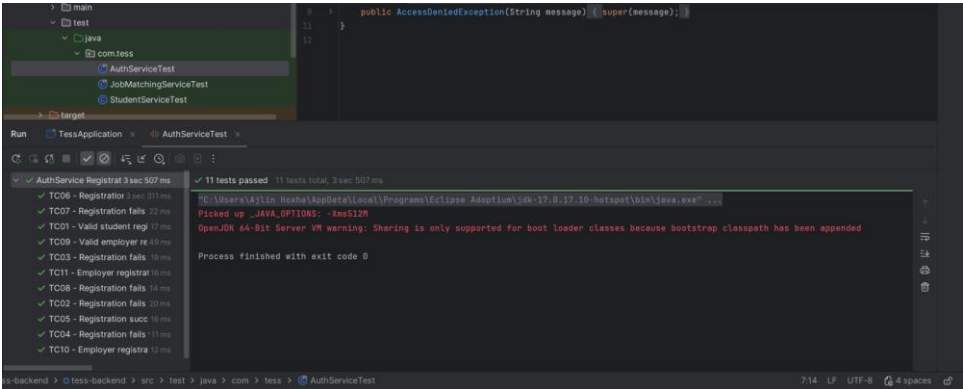
To run all tests, the following command was executed in the project root directory: mvn test.



Summary of Results

Component	Total Tests	Passed	Failed	Pass Rate
JobMatchingService	15	15	0	100%
AuthService (Register)	11	11	0	100%
StudentService (Availability)	9	9	0	100%
Total	35	35	0	100%

Screenshots of Results



7. Coverage Analysis and Reflection

JobMatchingService: The tests cover all three main methods (`hasScheduleConflict`, `hasSkillMatch`, `calculateCompatibilityScore`, `getMatchingJobsForStudent`). The cases include normal inputs, boundary conditions (back-to-back times, equal times), edge cases (no schedule defined, empty skills), and integration of the full pipeline. The most important paths: conflict detection and result ranking — are fully covered.

AuthService Registration: The tests cover all validation rules defined in the service: age range (16-35 inclusive), required fields (university, company name), active student confirmation and email uniqueness. Both boundary values of the age range are tested as separate cases.

StudentService Availability: The tests cover input validation (invalid time ranges), data integrity (old slots deleted before new ones), security (ownership check), and correct flag assignment (`isBusy`). The security test TC07 is particularly important as it tests the access control mechanism.

In the context of TESS, testing is especially important because the system handles sensitive user data, enforces authentication and authorization and implements a critical matching algorithm that directly affects the user experience. Errors in any of these areas could lead to security vulnerabilities, incorrect job matches or data integrity issues.

Conclusion

The testing phase validated three critical components of the TESS system: the job matching algorithm, the user registration validation and the availability schedule management. A total of 35 test cases were written and executed, all passing successfully.

The most significant finding from testing was confirming the correctness of the interval overlap formula in `JobMatchingService.hasScheduleConflict()`, particularly the boundary case where a job starts exactly when a class ends (back-to-back), which correctly returns no conflict. This edge case is important for student usability because it allows students to work immediately after their last class of the day.

The ownership check test in `StudentService` confirmed that the security mechanism preventing students from editing each other's profiles works correctly, which is important for data integrity and privacy in the platform.